

Express.js

Séance 4 - Notre backend devient une vraie API web

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

Routing - Middlewares - CRUD avec Prisma - axios (APIs externes) - Zod - Postman

Où on en est, où on va

Récap S1-S3 et objectifs S4

Acquis (S1, S2, S3)

- HTTP + REST + JSON (S1)
- JavaScript moderne + Node.js (S2)
- TypeScript de base, npm, .env (S3)
- Prisma + Neon + utils/prisma.ts
- CRUD via scripts (sans serveur web)

Objectifs de la séance

- Créer un vrai serveur web avec Express
- Définir des routes REST (GET, POST, PUT, DELETE)
- Utiliser les middlewares (logger, cors, erreurs)
- Brancher Prisma dans nos routes
- Appeler des APIs externes avec axios
- Valider les entrées (Zod) + tester (Postman)

L'esprit de la séance. Jusqu'ici, notre code Node tournait juste comme un script. À partir de maintenant, il devient un serveur web : il écoute des requêtes HTTP et y répond. C'est le moment où le frontend (à venir) pourra enfin se brancher à notre backend. On va bâtir une API REST complète qui parle à PostgreSQL via Prisma et qui peut aussi consommer des APIs externes.

PARTIE 1

Pourquoi Express ?

http natif vs Express

Pourquoi on n'écrit plus de serveur à la main

Avec http natif (vu en S2) :

```
import http from 'http';

const server = http.createServer((req, res) => {
  if (req.url === '/livres' && req.method === 'GET') {
    res.writeHead(200, {
      'Content-Type': 'application/json'
    });
    res.end(JSON.stringify([]));
  } else {
    res.writeHead(404);
    res.end('Not Found');
  }
});
server.listen(3000);
```

JS

Avec Express :

```
import express from 'express';

const app = express();

app.get('/livres', (req, res) => {
  res.json([]);
});

app.listen(3000);
```

JS

- **Routing déclaratif** : `app.get`, `app.post`, `app.put`, `app.delete`
- **Moins de boilerplate**, plus lisible, plus maintenable
- **Écosystème énorme** : milliers de middlewares prêts à l'emploi

Express en 1 minute

Le framework web le plus utilisé de Node.js

Express est un framework web minimaliste pour Node.js. Il fournit un système de routes, de middlewares, et beaucoup de petits outils pour bâtir une API REST rapidement, sans dicter une structure rigide.

Minimaliste

- Pas de dogmes, on choisit
- Très peu d'opinions
- Idéal pour apprendre

Énorme communauté

- Sorti en 2010, hyper mature
- Documentation complète
- Stack Overflow infini

Écosystème

- cors, helmet, morgan...
- Pas besoin de tout coder
- npm install et c'est plié

À retenir :

- **Express tourne SUR Node.js** : il utilise le module http en dessous
- **Express, ce n'est PAS magique** : ça reste du JavaScript classique
- Ce qu'on apprend ici se retrouvera dans Next.js (API Routes)

PARTIE 2

Notre premier serveur Express

Installation et structure

Préparer le projet Express + TypeScript

1. Installer Express et ses types :

```
npm install express
npm install -D @types/express
```

bash

2. Structure recommandée :

```
mon-backend/
src/
  server.ts      <- point d'entrée Express
  routes/
    livres.routes.ts
  controllers/
    livres.controller.ts
  utils/
    prisma.ts    <- vu en S3
  prisma/schema.prisma
  .env
  package.json
```

tree

Pourquoi cette structure ?

- routes/ : les URL et verbes
- controllers/ : la logique
- utils/ : code partagé
- Chaque dossier a un rôle clair
- Facile à maintenir à plusieurs

Hello World en Express

Le serveur le plus simple possible

```
// src/server.ts
import express, { Request, Response } from 'express';
import dotenv from 'dotenv';
dotenv.config();

const app = express();
app.use(express.json()); // parse le JSON entrant

app.get('/', (req: Request, res: Response) => {
  res.json({ message: 'Bonjour depuis Express !' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Serveur sur http://localhost:${PORT}`);
});
```

TS

Ligne par ligne

- import express
- + types Request, Response
-
- dotenv : charge .env
-
- app.use(express.json()) :
- lit les corps JSON
-
- app.get('/', ...): 1 route
-
- app.listen : démarre

Lancer le serveur et tester

Trois façons de vérifier que ça marche

1. Lancer (avec tsx, vu en S3) :

```
npx tsx watch src/server.ts      # auto-reload à chaque save  
# ou via package.json : npm run dev
```

bash

2. Tester (au choix) :

Navigateur

- `http://localhost:3000`
- Idéal pour les GET sans paramètres
- Affiche le JSON brut

curl (terminal)

- `curl http://localhost:3000`
- Pratique en CI / scripts
- Voir aussi : `-X POST`, `-d`, `-H`

Postman / Thunder

- Interface graphique
- Sauvegarde les requêtes
- Le meilleur pour développer

Astuce : ajoutez un script `"dev": "tsx watch src/server.ts"` dans `package.json`

PARTIE 3

Le routing : définir nos endpoints

Les méthodes HTTP avec Express

Un verbe + un chemin = une route

```
// LIRE : tous les livres
app.get('/livres', (req, res) => {
  res.json([]);
});

// CREER : un nouveau livre
app.post('/livres', (req, res) => {
  res.status(201).json({ id: 1 });
});

// REMPLACER : un livre entier
app.put('/livres/:id', (req, res) => { ... });

// MODIFIER PARTIELLEMENT
app.patch('/livres/:id', (req, res) => { ... });

// SUPPRIMER
app.delete('/livres/:id', (req, res) => { ... });
```

TS

Verbe	Sens
GET	Lire
POST	Créer
PUT	Remplacer
PATCH	Modifier
DELETE	Supprimer

Convention REST : un verbe HTTP par opération + une URL stable par ressource.

Paramètres d'URL et query string

req.params vs req.query

Paramètres d'URL (req.params) :

```
// Route avec :id
app.get('/livres/:id', (req, res) => {
  const id = Number(req.params.id);
  res.json({ id });
});

// Appel : GET /livres/42
// -> req.params.id = '42'
```

TS

Query string (req.query) :

```
// Route avec ?cle=valeur
app.get('/livres', (req, res) => {
  const { auteur, annee } = req.query;
  res.json({ auteur, annee });
});

// Appel : GET /livres?auteur=Hugo
// -> req.query.auteur = 'Hugo'
```

TS

- **req.params** : valeurs dans le CHEMIN, marquées par :nom
- **req.query** : valeurs OPTIONNELLES après ? dans l'URL
- **Les valeurs sont TOUJOURS des strings** (faire Number(...) si besoin)
- On utilise params pour identifier, query pour filtrer ou paginer

Le corps de la requête : req.body

Pour POST, PUT et PATCH

Pour lire un corps JSON, on active le middleware `express.json()` AU DÉBUT de l'application. Sans lui, `req.body` est undefined.

```
import express from 'express';
const app = express();

// ----- IMPORTANT -----
app.use(express.json());

app.post('/livres', (req, res) => {
  const { titre, auteur, annee } = req.body;
  console.log(req.body);
  res.status(201).json({ titre, auteur, annee });
});
```

TS

Requête envoyée

- POST /livres
- Content-Type: application/json
- {
- "titre": "Bel-Ami",
- "auteur": "Maupassant",
- "annee": 1885
- }

Oubli fréquent : pas de `express.json()` -> `req.body` est undefined. Si vous voyez ça en débuggant, c'est presque toujours la cause.

Envoyer une réponse

res.json, res.status, codes HTTP

```
// JSON simple
res.json({ ok: true });

// Code de statut + JSON
res.status(201).json(nouveau);

// Erreur 404
res.status(404).json({
  error: 'Livre introuvable'
});

// Pas de contenu (DELETE réussi)
res.status(204).end();

// Texte brut (rare)
res.send('Bonjour');
```

TS

Code	Quand
200	OK (GET, PUT)
201	Created (POST)
204	No Content (DELETE)
400	Bad Request
401	Unauthorized
404	Not Found
500	Erreur serveur

Astuce : un `res.status()` sans `.json()` ne suffit pas, il faut TOUJOURS terminer la réponse avec `.json()`, `.send()` ou `.end()`.

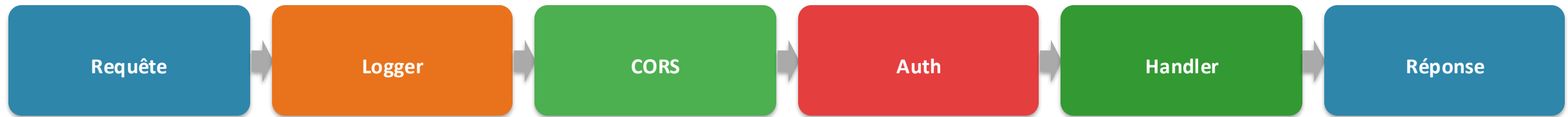
PARTIE 4

Les middlewares

C'est quoi un middleware ?

Une fonction qui intercepte la requête

Un middleware est une fonction qui reçoit (req, res, next). Elle peut lire la requête, la modifier, ou même y répondre. Elle appelle next() pour passer la main au middleware suivant (ou au handler final).



```
// Structure d'un middleware
app.use((req, res, next) => {
  // ... faire quelque chose (lire, modifier, vérifier)
  next();    // TRÈS IMPORTANT : passer au suivant
});
```

TS

Oublier next() -> la requête reste bloquée. Le client attend indéfiniment.

Middlewares essentiels

Intégrés + packages tiers à connaître

```
// Installation
npm install cors helmet morgan

// Activation
import cors from 'cors';
import helmet from 'helmet';
import morgan from 'morgan';

app.use(express.json()); // parse les corps JSON
app.use(cors()); // autorise les appels cross-origin
app.use(helmet()); // en-têtes de sécurité
app.use(morgan('dev')); // log coloré des requêtes
```

TS

Middleware	Rôle	Quand l'utiliser
express.json	Parse les corps JSON entrants	Toujours, dès qu'on accepte POST/PUT/PATCH
cors	Cross-Origin Resource Sharing	Quand le frontend est sur un autre port/domaine
helmet	Ajoute des en-têtes de sécurité	En production (XSS, clickjacking...)
morgan	Logger HTTP coloré	En développement (debug et monitoring)

Middleware custom + gestion d'erreurs

Logger maison + handler 4 paramètres

Notre propre logger :

```
app.use((req, res, next) => {  
  const start = Date.now();  
  res.on('finish', () => {  
    const ms = Date.now() - start;  
    console.log(  
      `${req.method} ${req.url}  
        ${res.statusCode} - ${ms}ms`  
    );  
  });  
  next();  
});
```

TS

Handler d'erreurs (4 params !) :

```
// À placer EN DERNIER  
app.use((err, req, res, next) => {  
  console.error(err);  
  res.status(500).json({  
    error: 'Erreur interne du serveur'  
  });  
});
```

TS

- **Le handler d'erreurs a 4 paramètres :** (err, req, res, next). C'est ainsi qu'Express le reconnaît.
- **Il doit être déclaré APRÈS toutes les routes**, à la toute fin de l'app.
- **Dans nos routes async :** on utilise **try/catch** ou on relaie l'erreur avec **next(err)**

PARTIE 5

Organiser le code : Router

express.Router : séparer les routes

Une ressource = un fichier

src/routes/livres.routes.ts :

```
import { Router } from 'express';
const router = Router();

router.get('/', (req, res) => {
  res.json([]);
});

router.get('/:id', (req, res) => {
  res.json({ id: req.params.id });
});

export default router;
```

TS

src/server.ts :

```
import express from 'express';
import livresRouter from
  './routes/livres.routes';

const app = express();
app.use(express.json());

// Toutes les routes ont préfixe /livres
app.use('/livres', livresRouter);

app.listen(3000);
```

TS

- **Un Router est un mini-app** : il a ses propres routes et middlewares
- On le branche avec `app.use('/préfixe', router)`
- **Conseil** : un fichier de routes par ressource (livres, emprunts, users)

Architecture : routes / controllers / services

Séparer le « quoi » du « comment »

Plus le projet grandit, plus on a intérêt à séparer les responsabilités. Le découpage classique en backend Node :

routes/

- Les URL et les verbes
- Branche chaque route à une méthode de contrôleur
- Ne contient PAS de logique

controllers/

- Reçoivent (req, res)
- Valident, formattent
- Appellent un service
- Renvoient la réponse

services/

- La logique métier pure
- Pas de req/res
- Appellent Prisma, axios...
- Réutilisables, testables

```
// routes -> controller -> service -> Prisma
router.get('/', getAllLivres);
async function getAllLivres(req, res) {
  const livres = await livreService.getAll();
  res.json(livres);
}
```

TS

PARTIE 6

Express + Prisma : le CRUD complet

Brancher Prisma à Express

On importe notre singleton

On réutilise le fichier `utils/prisma.ts` vu en S3. Aucune nouvelle configuration Prisma, juste un import.

```
// src/server.ts
import express from 'express';
import prisma from '../utils/prisma';    // <-- notre singleton de S3

const app = express();
app.use(express.json());
// ... routes ci-dessous
```

TS

Avantages du singleton

- Une seule instance partagée -> pas de surcharge de connexions
- Configuration centralisée (adapter Neon, logging) en un seul endroit
- Auto-complétion TS dans tout le projet
- Si on change de provider plus tard, un seul fichier à modifier

GET : lire les données

findMany + findUnique avec gestion d'erreur 404

TS

```
// GET /livres -> tous (ou filtrés via query)
app.get('/livres', async (req, res) => {
  const { auteur } = req.query;
  const livres = await prisma.livre.findMany({
    where: auteur ? { auteur: { contains: String(auteur) } } : undefined,
  });
  res.json(livres);
});

// GET /livres/:id -> un livre précis
app.get('/livres/:id', async (req, res) => {
  const id = Number(req.params.id);
  const livre = await prisma.livre.findUnique({ where: { id } });

  if (!livre) {
    return res.status(404).json({ error: 'Livre introuvable' });
  }
  res.json(livre);
});
```

POST : créer une ressource

Lire req.body et renvoyer 201

```
// POST /livres -> créer un livre
app.post('/livres', async (req, res) => {
  try {
    const { titre, auteur, annee } = req.body;

    // Validation très basique
    if (!titre || !auteur || !annee) {
      return res.status(400).json({
        error: 'Champs requis manquants'
      });
    }

    const nouveau = await prisma.livre.create({
      data: { titre, auteur, annee }
    });
    res.status(201).json(nouveau);
  } catch (err) {
    res.status(500).json({ error: 'Erreur serveur' });
  }
});
```

TS

Points à retenir

- try/catch : indispensable
- avec Prisma (async)
- Validation minimale ici, on verra Zod plus tard
- Statut 201 : Created (pas 200, c'est plus précis)
- Renvoie l'objet créé : utile côté client (id, dates...)

PUT + DELETE et erreurs Prisma

Gérer le cas « pas trouvé »

TS

```
// PUT /livres/:id -> modifier
app.put('/livres/:id', async (req, res) => {
  try {
    const id = Number(req.params.id);
    const livre = await prisma.livre.update({
      where: { id },
      data: req.body,
    });
    res.json(livre);
  } catch (err: any) {
    if (err.code === 'P2025') { // Prisma : record not found
      return res.status(404).json({ error: 'Livre introuvable' });
    }
    res.status(500).json({ error: 'Erreur serveur' });
  }
});

// DELETE /livres/:id -> supprimer
app.delete('/livres/:id', async (req, res) => {
  await prisma.livre.delete({ where: { id: Number(req.params.id) } });
  res.status(204).end();
});
```

PARTIE 7

Express + axios : consommer des APIs

axios : appeler une API depuis Node

Comme fetch, mais plus pratique

Installation :

```
npm install axios
```

bash

```
import axios from 'axios';

// GET
const { data } = await axios.get(
  'https://api.adviceslip.com/advice'
);
console.log(data.slip.advice);

// POST avec un corps
const res = await axios.post(
  'https://api.exemple.com/items',
  { name: 'Livre' },
  { headers: { Authorization: 'Bearer ...' } }
);
```

TS

axios vs fetch

- axios renvoie déjà du JSON
- (pas besoin de .json())
-
- Erreurs HTTP -> exception
- (plus simple à attraper)
-
- Headers et auth pratiques
-
- Timeouts et intercepteurs
- intégrés

Exemple complet : Express + Prisma + axios

Une route qui combine BD et API externe

TS

```
// GET /livres/:id/enrichi
// -> retourne le livre + une citation aléatoire (API externe)
app.get('/livres/:id/enrichi', async (req, res) => {
  try {
    const id = Number(req.params.id);

    // 1) Lire le livre en BD via Prisma
    const livre = await prisma.livre.findUnique({ where: { id } });
    if (!livre) {
      return res.status(404).json({ error: 'Livre introuvable' });
    }

    // 2) Appeler une API externe via axios
    const { data } = await axios.get('https://api.adviceslip.com/advice');
    const citation = data.slip.advice;

    // 3) Combiner et répondre
    res.json({ ...livre, citation });
  } catch (err) {
    res.status(500).json({ error: 'Erreur serveur' });
  }
});
```

PARTIE 8

Validation, tests, bonnes pratiques

Zod : valider req.body proprement

Plus de if (!champ) partout

Installation :

```
npm install zod
```

bash

```
import { z } from 'zod';

// 1) Décrire ce qu'on attend
const LivreSchema = z.object({
  titre: z.string().min(1),
  auteur: z.string().min(1),
  annee: z.number().int().positive(),
});

// 2) Valider req.body
app.post('/livres', async (req, res) => {
  const parsed = LivreSchema.safeParse(req.body);
  if (!parsed.success) {
    return res.status(400).json({ errors: parsed.error.issues });
  }
  const livre = await prisma.livre.create({ data: parsed.data });
  res.status(201).json(livre);
});
```

TS

Tester avec Postman

L'outil indispensable du développeur backend

Postman (ou Thunder Client dans VS Code) permet d'envoyer toute requête HTTP imaginable et de visualiser la réponse. Aucune frontend n'est nécessaire pour tester son backend.

Ce qu'on fait avec Postman

- Choisir le verbe HTTP
- Saisir l'URL (`http://localhost:3000/...`)
- Ajouter des en-têtes
- Coller un corps JSON
- Voir le statut, le temps, le JSON

Bonnes pratiques

- Créer une « Collection » par projet
- Utiliser des variables (`baseUrl`)
- Sauvegarder vos requêtes types
- Exporter vers l'équipe (.json)
- Tests automatiques (Postman scripts)

Requête type

GET /livres

POST /livres

DELETE /livres/1

Configuration

URL seule, pas de corps

Body raw JSON : `{ "titre": "..."` }

URL seule, pas de corps

Bonnes pratiques à garder en tête

Petit guide de l'API REST propre

Routes

- Pluriels : /livres pas /livre
- Pas de verbes : pas /getLivres
- Imbrication propre :
 - /livres/:id/emprunts

Codes HTTP

- 200 lecture OK
- 201 création
- 204 suppression OK
- 400 mauvaise requête
- 404 introuvable
- 500 erreur serveur

Sécurité

- Toujours valider req.body
- Variables sensibles -> .env
- Helmet + cors
- Jamais de secrets dans Git

Code

- try/catch sur tout async
- Erreurs centralisées (middleware)
- Une route -> une responsabilité
- Séparer routes / controllers / services
- TypeScript : on profite des types Prisma

Dev quotidien

- npm run dev avec tsx watch
- Postman pour tester chaque route
- morgan pour voir les requêtes
- Prisma Studio pour la BD
- Commits petits et fréquents

Récapitulatif de la séance

Ce qu'on a construit aujourd'hui

Express

- app.get / post / put...
- req.params, req.query,
- req.body, res.json
- express.json()

Middleware

- concept (req, res, next)
- cors, helmet, morgan
- logger custom
- handler d'erreurs

Express + Prisma

- CRUD complet REST
- findMany, findUnique
- create, update, delete
- erreurs Prisma (P2025)

axios + Zod + Postman

- consommer des APIs
- valider req.body
- tester proprement
- bonnes pratiques REST

Les commandes à mémoriser :

```
npm install express cors helmet morgan dotenv axios zod
npm install -D @types/express @types/cors @types/morgan
```

```
# scripts package.json
"dev": "tsx watch src/server.ts"
"start": "node dist/server.js"
```

bash

Pour la prochaine séance

On consolide et on prépare la suite

À faire de votre côté

- Refaire le CRUD livres avec Express
- Tester chaque route dans Postman
- Ajouter Zod pour POST + PUT
- Ajouter cors, morgan, helmet
- Brancher au moins UNE API externe (axios)
- Pousser le tout sur GitHub

Au programme ensuite

- Relations dans l'API (Livre + Emprunt)
- Pagination, filtres, tri
- Authentification (premiers pas avec Clerk)
- Logs structurés et monitoring
- Puis : React (frontend) -> Next.js full-stack
- Et finalement : déploiement (Vercel)

Vous avez maintenant une vraie API REST testable depuis Postman.

C'est exactement ce dont un frontend (React, mobile, etc.) a besoin pour exister.

Questions ?

On a posé beaucoup de briques : prenons le temps de pratiquer.

Construisez, cassez, corrigez, documentez et déployez.